REVISION CHEAT SHEET:

SQL Server Security

Complete notes - authentication - authorization - roles - permissions - RLS - encryption - SQL injection - least privilege

INDEX - Main Topics & Important Subtopics

Study flow: identity -> login/user -> roles and permissions -> row/display controls -> encryption -> injection defense -> least privilege and layered security.

- Authentication
- Authorization
- SQL Server Login
- Database User
- Roles
- Server Roles
- Database Roles
- Permissions and Securables
- GRANT / DENY / REVOKE
- Schema Permissions
- Object Permissions
- Row-Level Security (RLS)
- Dynamic Data Masking (DDM)
- Always Encrypted
- Transparent Data Encryption (TDE)
- TLS and Encryption in Transit
- SQL Injection
- Parameterized Queries
- Least Privilege
- Defense in Depth
- Common Security Mistakes
- Practical FundraisingDb Security
- Interview Questions
- Final Security Mental Model
- Final Memory Rules

SQL Server security is the set of features that controls **who can connect to SQL Server, what they are allowed to do after connecting, which rows and columns they are allowed to see, and how sensitive data is protected from unauthorized access**.

Security is not only about creating a username and password. A real production database normally needs several layers of protection working together.

For example, imagine your `FundraisingDb` is used by an ASP.NET Core Web API.

The Web API may need to:

- read Constituents,
- create Gifts,
- create GiftAllocations,
- update Tasks,
- read Campaigns and Funds.

But the Web API probably does **not** need permission to:

- drop the database,
- create SQL Server logins,
- change SQL Server configuration,
- read every database on the server,
- disable security,
- become `sysadmin`.

That is the central idea behind SQL Server security:

> ! **Give each identity only the access it actually needs, protect sensitive information at several layers, and never depend on only one security feature.**

A simple security flow is:

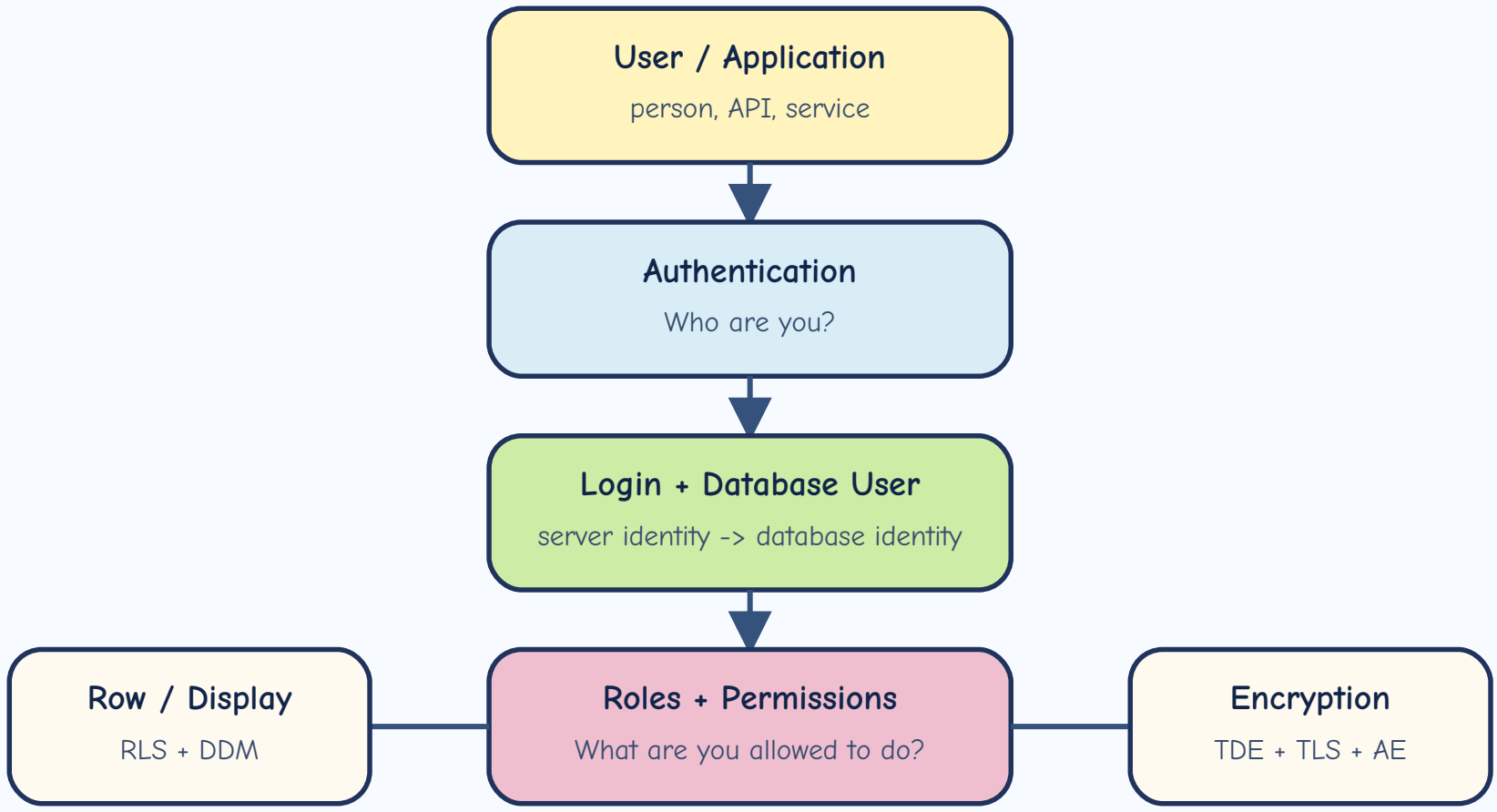
EXAMPLE / FLOW

```
User / Application
        |
        v
Authentication
"Who are you?"
        |
        v
Login / Identity
        |
        v
Database User
        |
        v
Role
        |
        v
Permissions
"What are you allowed to do?"
        |
        +--------------------------+
        |                          |
        v                          v
Row / Column Protection       Encryption
RLS / Masking                TDE / TLS /
                             Always Encrypted
```

Microsoft recommends role separation and assigning only the minimum permissions applications and users require.

DIAGRAM

SQL Server Security - Layered Access Flow



Authentication establishes identity; authorization and data-protection layers then control what can be reached and how it is protected.

Authentication

What authentication really means

Authentication is the process SQL Server uses to verify the identity of a person, application, service, or process that is trying to connect to SQL Server.

In plain words:

> ! Before SQL Server decides what you are allowed to do, it first needs to know who you are.

Imagine the entrance to an office building.

A person walks toward the entrance.

Security first checks:

EXAMPLE / FLOW

```
Who are you?

Do you have a valid employee badge?

Is this badge real?

Are you allowed to enter the building?
```

That step is authentication.

After the person enters, another system decides:

EXAMPLE / FLOW

```
Can you enter the finance room?

Can you enter the server room?

Can you see payroll files?
```

That second part is authorization.

So remember:

EXAMPLE / FLOW

```
Authentication
→ Who are you?

Authorization
→ What are you allowed to do?
```

SQL Server authentication scenario

Imagine your ASP.NET Core Web API wants to connect to:

EXAMPLE / FLOW

```
SQL Server
    |
    v
FundraisingDb
```

The application knows:

EXAMPLE / FLOW

```
Server name
Database name
```

but knowing the server name does not mean SQL Server should allow the connection.

The application needs an identity.

Conceptually:

EXAMPLE / FLOW

```
ASP.NET Core API
       |
       | tries to connect
       v
SQL Server
       |
       | verifies application identity
       v
Authentication succeeds?
       |
    +--+--+
    |     |
   Yes    No
    |     |
    v     v
continue  reject connection
```

Common authentication methods

SQL Server environments can authenticate identities using mechanisms such as:

- Windows authentication,
- Microsoft Entra identities where supported/configured,
- SQL Server authentication.

A Windows or directory-based identity lets authentication be managed through an enterprise identity system.

A SQL Server login can instead contain a SQL Server-managed username/password.

For enterprise systems, centrally managed identities and groups are often easier to secure and audit than distributing independent SQL passwords.

Authorization

Definition

Authorization is the process of deciding what an authenticated identity is allowed to do after SQL Server knows who that identity is.

Authentication may successfully tell SQL Server:

EXAMPLE / FLOW

```
This is FundraisingApiLogin
```

But SQL Server still needs to decide:

EXAMPLE / FLOW

```
Can this identity read Gift?

Can it insert Gift?

Can it delete Gift?

Can it execute fundraising.usp_CreateGift?

Can it create another login?

Can it drop FundraisingDb?
```

Those decisions are authorization.

Authorization in SQL Server is mainly implemented through:

EXAMPLE / FLOW

```
Users

Roles

GRANT

DENY

REVOKE

Permissions
```

SQL Server Login

Complete definition

A **login is a server-level security principal that represents an identity allowed to authenticate to a SQL Server instance**.

The easiest way to understand a login is:

EXAMPLE / FLOW

```
Login
=
Identity used to enter the SQL Server instance
```

The login exists at the SQL Server instance level rather than belonging to one particular application database. Microsoft describes logins as identities used to sign in to the SQL Server Database Engine.

Real-world example

Suppose your SQL Server instance contains:

EXAMPLE / FLOW

```
SQL Server Instance
    |
    +-- FundraisingDb
    +-- ReportingDb
    +-- AuditDb
```

Your application uses:

EXAMPLE / FLOW

```
FundraisingApiLogin
```

The login exists at:

EXAMPLE / FLOW

```
SQL Server Instance level
```

not inside only `FundraisingDb`.

Conceptually:

EXAMPLE / FLOW

```
SQL Server Instance
        |
        +-- Login: FundraisingApiLogin
        |
        +-- FundraisingDb
        |
        +-- ReportingDb
```

Creating a SQL Server login

Learning example:

SQL

```
CREATE LOGIN FundraisingApiLogin
WITH PASSWORD = '<strong-password>';
```

This creates the server-level identity.

But creating a login **does not automatically mean the login can access every table in `FundraisingDb`**.

The login normally needs a corresponding database user.

Windows login example

```sql
CREATE LOGIN [CONTOSO\FundraisingUsers]
FROM WINDOWS;
```

This could represent:

```
Windows user
or
Windows security group
```

Using groups can simplify administration because permissions can be assigned to the group rather than to every individual employee.

Why login and database user are separate

Suppose one SQL Server contains:

```
FundraisingDb
AccountingDb
HRDb
```

The same login might need:

```
Full read access in FundraisingDb

No access to HRDb

Limited read access in AccountingDb
```

Separating:

```
Login
```

from:

```
Database User
```

makes that possible.

When to use a login

Use a login when an identity needs to authenticate to the SQL Server instance.

Examples:

EXAMPLE / FLOW

```
Web API service identity

Application service account

DBA

Reporting service

Administrator
```

What not to do

Do not let normal applications connect using:

EXAMPLE / FLOW

```
sa

sysadmin account

developer personal administrator account
```

unless there is an exceptional administrative reason.

That violates least privilege and greatly increases the damage that an application compromise could cause.

Database User

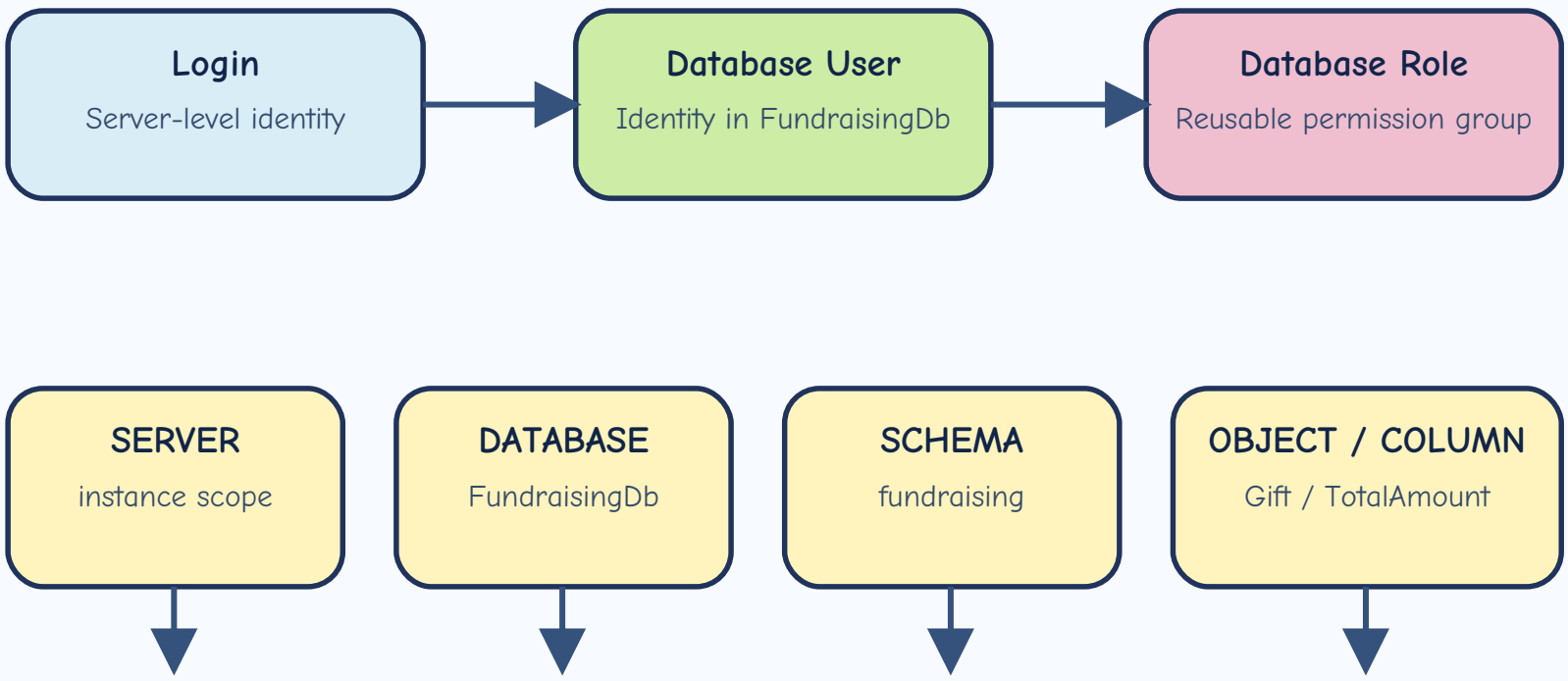
Complete definition

A **database user is a database-level security principal that represents an identity inside one specific database and receives permissions within that database**.

Simple memory rule:

DIAGRAM

Login, User, Role, and Permission Scope



Keep server identity, database identity, reusable roles, and securable scope conceptually separate.

EXAMPLE / FLOW

```
LOGIN
→ gets identity into SQL Server

USER
→ gives that identity a security identity
  inside a database
```

Mapping a login to a database user

First:

SQL

```sql
CREATE LOGIN FundraisingApiLogin
WITH PASSWORD = '<strong-password>';
```

Then:

SQL

```sql
USE FundraisingDb;
GO

CREATE USER FundraisingApiUser
FOR LOGIN FundraisingApiLogin;
```

Now the relationship is:

EXAMPLE / FLOW

```
SQL Server Instance
        |
        v
FundraisingApiLogin
        |
        | maps to
        v
FundraisingDb
        |
        v
FundraisingApiUser
```

Why a user is needed

Suppose SQL Server recognizes:

EXAMPLE / FLOW

```
FundraisingApiLogin
```

during connection.

Inside **FundraisingDb**, SQL Server then evaluates permissions belonging to:

EXAMPLE / FLOW

```
FundraisingApiUser
```

and its roles.

That user may have:

EXAMPLE / FLOW

```
SELECT on Constituent

INSERT on Gift

UPDATE on Task
```

but not:

EXAMPLE / FLOW

```
DROP TABLE

ALTER DATABASE

CREATE LOGIN
```

Login vs User

A beginner should understand this clearly:

Login	Database User
Exists at SQL Server instance level	Exists inside a database
Used for authentication	Used for database authorization
Gets you into SQL Server	Determines permissions in a database
Can map to multiple database users	Belongs to one database
Added to server roles	Added to database roles

The easiest interview memory rule is:

> ! **Login = server identity. User = database identity.**

Roles

What is a role?

A **role is a security group that contains principals and receives permissions so that permissions do not have to be granted individually to every user**.

Imagine five reporting users:

EXAMPLE / FLOW

```
Sarah
John
Alex
Priya
David
```

You could do:

EXAMPLE / FLOW

```
GRANT SELECT to Sarah
GRANT SELECT to John
GRANT SELECT to Alex
GRANT SELECT to Priya
GRANT SELECT to David
```

But this becomes difficult to manage.

A better design is:

EXAMPLE / FLOW

```
ReportingRole
      |
      +-- Sarah
      +-- John
      +-- Alex
      +-- Priya
      +-- David
```

Then:

SQL

```
GRANT SELECT
ON SCHEMA::fundraising
TO ReportingRole;
```

Now all role members receive the permission.

Microsoft recommends grouping principals into roles and granting permissions to roles rather than repeatedly granting individual permissions.

Server Roles

Definition

A **server role is a role whose permissions operate at SQL Server instance scope**.

Server roles generally contain:

EXAMPLE / FLOW

```
Logins
```

rather than normal database users.

Think:

EXAMPLE / FLOW

```
Server Role
→ controls SQL Server instance activities
```

Examples:

EXAMPLE / FLOW

```
Manage logins

Create databases

View server state

Change server configuration

Perform administrative operations
```

Important fixed server roles

sysadmin

`sysadmin` is the most powerful fixed server role.

Members can perform essentially any SQL Server activity.

Important rule:

> ! Permissions cannot normally be denied to a `sysadmin` member.

That is why applications should almost never run as `sysadmin`.

serveradmin

Can change important server-wide configuration and shut down the server.

securityadmin

Can manage logins and permissions.

This role is extremely powerful because someone who controls security can often escalate privileges. Microsoft specifically recommends treating it as highly privileged and suggests newer granular server roles where appropriate.

processadmin

Can manage SQL Server processes.

bulkadmin

Supports bulk operations.

This is still a privileged capability and should be assigned using least privilege.

dbcreator

Can create, alter, drop, and restore databases.

public

Every login automatically belongs to the `public` server role.

You cannot remove this membership.

Therefore:

> ! If you grant a powerful permission to **public**, you may be giving it to every login.

Microsoft recommends giving **public** access only when you genuinely intend all users to receive it.

Modern server roles

Starting with SQL Server 2022, SQL Server includes additional fixed server roles prefixed with:

EXAMPLE / FLOW

```
##MS_
```

designed to support more granular least-privilege administration. Examples include roles for login management, database connection, and server-state visibility.

This is useful because previously organizations sometimes granted overly broad roles just to provide one administrative capability.

Custom server role

You can create your own server role:

SQL

```
CREATE SERVER ROLE MonitoringOperators;
```

Then:

SQL

```
GRANT VIEW SERVER STATE
TO MonitoringOperators;
```

Add a login:

SQL

```
ALTER SERVER ROLE MonitoringOperators
ADD MEMBER MonitoringLogin;
```

This is often better than granting a much broader built-in administrative role.

Database Roles

Definition

A **database role is a role that groups permissions inside one specific database**.

Database roles contain:

EXAMPLE / FLOW

```
database users
```

and are ideal for application-level security.

Example:

EXAMPLE / FLOW

```
FundraisingDb
      |
      +-- FundraisingApiRole
      |
      +-- ReportingRole
      |
      +-- GiftProcessingRole
```

Important fixed database roles

db_owner

Has very broad control over the database.

A normal Web API should generally not need this.

db_datareader

Can read user tables and views throughout the database.

Convenient, but broad.

If the application only needs a few objects, a custom role is safer.

db_datawriter

Can insert, update, and delete data from user tables.

Again, useful but broad.

db_ddladmin

Can perform database DDL operations such as creating or altering objects.

Do not give this to normal applications unless schema modification is genuinely part of the application responsibility.

db_backupoperator

Can perform database backups.

db_securityadmin

Can manage database permissions.

This is powerful and should be tightly controlled.

public

Every database user belongs to `public`.

As with server `public`, avoid granting sensitive permissions here.

Custom Database Roles

For applications, custom database roles are usually easier to secure.

Example:

```sql
USE FundraisingDb;
GO

CREATE ROLE FundraisingApiRole;
```

Add user:

```sql
ALTER ROLE FundraisingApiRole
ADD MEMBER FundraisingApiUser;
```

Now permissions can be assigned to the role:

```sql
GRANT SELECT
ON OBJECT::constituent.Constituent
TO FundraisingApiRole;

GRANT SELECT, INSERT
ON OBJECT::fundraising.Gift
TO FundraisingApiRole;
```

Server Role vs Database Role

```
Server Role
→ SQL Server instance level

Database Role
→ one database
```

Example:

```
##MS_ServerStateReader##
→ server-wide monitoring capability

FundraisingApiRole
→ permissions inside FundraisingDb
```

Permissions

A permission answers:

> ! What action can this principal perform on this SQL Server resource?

Examples:

EXAMPLE / FLOW

```
SELECT

INSERT

UPDATE

DELETE

EXECUTE

ALTER

CONTROL

VIEW DEFINITION
```

A permission normally involves three things:

EXAMPLE / FLOW

```
Principal
+
Permission
+
Securable
```

Example:

EXAMPLE / FLOW

```
Principal:
ReportingRole

Permission:
SELECT

Securable:
fundraising.Gift
```

Securables

A **securable is something SQL Server protects with permissions**.

Examples:

EXAMPLE / FLOW

```
Server
Database
Schema
Table
View
Stored Procedure
Function
Column
```

Simplified hierarchy:

EXAMPLE / FLOW

```
SERVER
   |
   v
DATABASE
   |
   v
SCHEMA
   |
   v
OBJECT
   |
   v
COLUMN
```

Higher-scope permissions can imply access to securables underneath them. Microsoft documents SQL Server permissions as a hierarchy of securables.

GRANT

Complete definition

GRANT explicitly gives a principal permission to perform an operation on a securable.

Example:

SQL

```
GRANT SELECT
ON OBJECT::fundraising.Gift
TO ReportingRole;
```

This means:

EXAMPLE / FLOW

```
ReportingRole
is allowed to SELECT
fundraising.Gift
```

More examples

Allow inserts:

SQL

```
GRANT INSERT
ON OBJECT::fundraising.Gift
TO GiftProcessingRole;
```

Allow task updates:

SQL

```
GRANT UPDATE
ON OBJECT::engagement.Task
TO FundraisingApiRole;
```

Allow stored procedure execution:

SQL

```
GRANT EXECUTE
ON OBJECT::fundraising.usp_CreateGift
TO FundraisingApiRole;
```

DENY

Complete definition

DENY explicitly prevents a principal from receiving a permission.

Example:

SQL

```
DENY DELETE
ON OBJECT::fundraising.Gift
TO ReportingRole;
```

Now that role is explicitly prevented from deleting Gift rows.

The important conceptual distinction is:

EXAMPLE / FLOW

```
GRANT
→ explicit YES

DENY
→ explicit NO
```

Why DENY exists

Suppose a user belongs to:

EXAMPLE / FLOW

```
GeneralFundraisingRole
```

which grants broad permissions.

But one special user must never delete Gifts.

You could explicitly deny:

SQL

```
DENY DELETE
ON OBJECT::fundraising.Gift
TO LimitedUser;
```

A denial generally overrides grants obtained through ordinary role membership.

`sysadmin` is an important exception because ordinary `DENY` does not restrict `sysadmin`.

REVOKE

Complete definition

`REVOKE` removes a previously explicit permission decision.

It does **not** mean:

EXAMPLE / FLOW

```
DENY
```

Example:

SQL

```
REVOKE SELECT
ON OBJECT::fundraising.Gift
FROM ReportingUser;
```

This removes that direct permission.

But the user may still be able to SELECT if they belong to:

EXAMPLE / FLOW

```
ReportingRole
```

which itself has SELECT.

So:

EXAMPLE / FLOW

```
REVOKE
→ remove explicit rule

DENY
→ explicit prohibition
```

Microsoft specifically warns that using **DENY** when you actually mean to remove a **GRANT** can create confusing permission behavior; use **REVOKE** for removing the explicit permission.

GRANT vs DENY vs REVOKE

Command	Meaning
GRANT	Give permission
DENY	Explicitly block permission
REVOKE	Remove explicit GRANT/DENY

Easy memory:

EXAMPLE / FLOW

```
GRANT
→ YES

DENY
→ NO

REVOKE
→ remove my previous answer
```

Schema Permissions

What is a schema?

A schema is a logical container for database objects.

Your **FundraisingDb** already uses meaningful schemas:

EXAMPLE / FLOW

```
core

constituent

fundraising

engagement

integration
```

For example:

EXAMPLE / FLOW

```
fundraising
    |
    +-- Gift
    +-- GiftAllocation
    +-- Campaign
    +-- Fund
```

What is a schema permission?

A **schema permission applies to the schema and, by implication, relevant objects contained inside that schema.**

Instead of granting SELECT separately:

EXAMPLE / FLOW

```
Gift
GiftAllocation
Campaign
Fund
```

you could write:

SQL

```
GRANT SELECT
ON SCHEMA::fundraising
TO ReportingRole;
```

Schema permissions can simplify permission management across many objects.

Example scenario

Reporting team needs read access to all fundraising reporting objects.

Instead of:

SQL

```
GRANT SELECT ON fundraising.Gift ...
GRANT SELECT ON fundraising.GiftAllocation ...
GRANT SELECT ON fundraising.Campaign ...
GRANT SELECT ON fundraising.Fund ...
```

one schema grant may be simpler:

SQL

```
GRANT SELECT
ON SCHEMA::fundraising
TO FundraisingReportingRole;
```

When schema permissions are useful

Use schema permissions when:

- many objects belong to the same security boundary,
- many users need the same kind of access,
- you want simpler administration.

When not to use broad schema permission

Do not use:

```sql
GRANT SELECT
ON SCHEMA::constituent
TO SomeRole;
```

if that role should only access one table.

Schema permissions also apply to future relevant objects created inside the schema, so broad schema permissions should be intentional.

Object Permissions

Definition

An **object permission grants or denies an action on one specific database object**.

Example:

```sql
GRANT SELECT
ON OBJECT::fundraising.Gift
TO GiftReportingRole;
```

This applies to:

```
fundraising.Gift
```

not automatically to:

```
fundraising.Fund

fundraising.Campaign
```

Typical table object permissions

Tables commonly use:

EXAMPLE / FLOW

```
SELECT
INSERT
UPDATE
DELETE
REFERENCES
```

Stored procedures commonly use:

EXAMPLE / FLOW

```
EXECUTE
```

Views may use:

EXAMPLE / FLOW

```
SELECT
```

Functions may use permissions appropriate to how they are invoked.

Microsoft documents object-level permissions such as SELECT, INSERT, UPDATE, DELETE, EXECUTE, ALTER, and CONTROL.

Schema Permission vs Object Permission

Use object-level permission when access should be narrow.

Example:

EXAMPLE / FLOW

```
Reporting user needs only Gift
```

Use:

SQL

```
GRANT SELECT
ON OBJECT::fundraising.Gift
TO GiftReportingRole;
```

Use schema permission when the whole schema is the intended boundary.

Example:

EXAMPLE / FLOW

```
Reporting system needs all fundraising objects
```

Use:

```sql
GRANT SELECT
ON SCHEMA::fundraising
TO FundraisingReportingRole;
```

Row-Level Security

Complete definition

Row-Level Security, or RLS, is a SQL Server feature that automatically limits which rows of a table a user or application can read or modify based on a security rule.

Normal table permission answers:

> ! Can this user SELECT from `fundraising.Gift`?

RLS answers:

> ! Which Gift rows can this user see?

That difference is extremely important.

Multi-tenant FundraisingDb example

Your tables contain:

```
TenantId
```

Suppose:

```
Tenant 1
→ ABC Children's Foundation

Tenant 2
→ XYZ Foundation
```

Without RLS, this query:

```sql
SELECT *
FROM fundraising.Gift;
```

could theoretically return both tenants' rows if the user has normal SELECT permission.

With correctly designed RLS:

EXAMPLE / FLOW

```
Tenant 1 session
→ only Tenant 1 rows

Tenant 2 session
→ only Tenant 2 rows
```

even though both use the same table.

How RLS works

RLS normally uses two objects:

EXAMPLE / FLOW

```
Predicate Function

Security Policy
```

Microsoft describes RLS as using an inline table-valued function as a security predicate attached to a table through a security policy.

Flow:

EXAMPLE / FLOW

```
Application
      |
      v
SELECT FROM fundraising.Gift
      |
      v
RLS Security Policy
      |
      v
Predicate Function
      |
      v
Allowed rows only
```

Filter predicate

A **filter predicate determines which existing rows the user is allowed to access.**

Example concept:

EXAMPLE / FLOW

```
Session TenantId = 1

Gift TenantId = 1
→ visible

Gift TenantId = 2
→ filtered out
```

Microsoft notes that filter predicates affect reading and row access for SELECT, UPDATE, and DELETE; filtered rows behave as though they are unavailable.

Learning RLS example

An application may securely establish tenant identity:

SQL

```
EXEC sys.sp_set_session_context
    @key = N'TenantId',
    @value = 1,
    @read_only = 1;
```

Then a predicate function could conceptually check the current tenant.

SQL

```
CREATE FUNCTION security.fn_TenantPredicate
(
    @TenantId BIGINT
)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
(
    SELECT 1 AS Allowed
    WHERE @TenantId =
          TRY_CONVERT
          (
              BIGINT,
              SESSION_CONTEXT(N'TenantId')
          )
);
```

Security policy:

SQL

```
CREATE SECURITY POLICY security.GiftTenantPolicy

ADD FILTER PREDICATE
    security.fn_TenantPredicate(TenantId)
    ON fundraising.Gift

WITH (STATE = ON);
```

Now ordinary queries can automatically respect tenant filtering.

Block predicates

A **block predicate prevents a write operation that would violate the RLS rule**.

Imagine Tenant 1 session tries:

EXAMPLE / FLOW

```
INSERT Gift
TenantId = 2
```

A properly designed block predicate can reject the insert.

SQL Server RLS supports block predicates around INSERT, UPDATE, and DELETE operations.

Why RLS is useful

RLS is valuable when the database itself should enforce row separation.

Examples:

EXAMPLE / FLOW

```
Multi-tenant SaaS

Regional data access

Employee can see only assigned records

Department-based visibility
```

For your schema, multi-tenant isolation is the most obvious example.

Advantages of RLS

RLS provides:

- centralized row filtering,
- less risk that a developer forgets `WHERE TenantId = ...`,
- consistent behavior across many queries,
- another database security layer.

RLS risks and limitations

RLS also adds complexity.

Poorly designed security predicates can:

- make queries harder to troubleshoot,
- affect execution plans,

- increase CPU,
- accidentally expose or hide rows.

The application must also establish tenant identity from a **trusted authenticated source**.

If the client is allowed to arbitrarily say:

EXAMPLE / FLOW

```
My TenantId = 2
```

then RLS can be defeated logically.

Dynamic Data Masking

Complete definition

Dynamic Data Masking, or DDM, is a SQL Server feature that changes how sensitive column values are displayed to users who do not have permission to see the full value, while keeping the original value unchanged in the database.

This means:

EXAMPLE / FLOW

```
Stored value:
john.smith@example.com
```

but an unauthorized user may see:

EXAMPLE / FLOW

```
jXXX@XXXX.com
```

The database still stores:

EXAMPLE / FLOW

```
john.smith@example.com
```

Microsoft explains that DDM masks query results without changing the underlying stored value.

FundraisingDb example

`constituent.Constituent` *contains:*

EXAMPLE / FLOW

```
PrimaryEmail

PrimaryPhone
```

Suppose reporting employees need to know:

EXAMPLE / FLOW

```
The constituent has an email
```

but do not need the full address.

A mask might be applied to `PrimaryEmail`.

Conceptual example:

SQL

```
ALTER TABLE constituent.Constituent

ALTER COLUMN PrimaryEmail

ADD MASKED WITH
(
    FUNCTION = 'email()'
);
```

Unauthorized user:

EXAMPLE / FLOW

```
John Smith
jXXX@XXXX.com
```

Authorized user:

EXAMPLE / FLOW

```
John Smith
john.smith@example.com
```

UNMASK

Users with appropriate:

EXAMPLE / FLOW

```
UNMASK
```

permission can see original values.

Important:

! `UNMASK` does not itself give SELECT permission.

The user still needs access to the object.

Common masking functions

SQL Server provides masking options such as:

EXAMPLE / FLOW

```
default()

email()

partial()

random()
```

DDM is not encryption

This is one of the most important rules.

EXAMPLE / FLOW

```
Dynamic Data Masking
→ changes what user sees

Encryption
→ cryptographically transforms data
```

A user with sufficiently powerful permissions may still be able to see the original value.

Microsoft explicitly notes that users with powerful CONTROL permissions can see unmasked data and that DDM is not intended to protect against every malicious privileged user.

When DDM is useful

Good scenarios:

EXAMPLE / FLOW

```
Support users

Call center employees

Test environments

Reports

Developers who do not need actual PII
```

When DDM should not be considered sufficient

Do not use masking as your only protection when you need:

EXAMPLE / FLOW

```
Cryptographic confidentiality

Protection against stolen files

Protection from highly privileged DB administrators

Network encryption
```

Always Encrypted

Complete definition

Always Encrypted is a SQL Server feature that encrypts selected sensitive columns so that the SQL Server Database Engine normally stores and processes encrypted values while the plaintext and encryption keys remain under client-side control.

In simple words:

! SQL Server can store the sensitive value without necessarily being able to read the original plaintext.

Microsoft documents Always Encrypted as using client-controlled encryption keys, with column master keys stored outside the database system.

Why Always Encrypted exists

TDE protects database files if somebody steals the storage.

But imagine an administrator has legitimate SQL access:

SQL

```
SELECT PrimaryEmail
FROM constituent.Constituent;
```

TDE transparently decrypts the page so SQL Server can return the plaintext.

Always Encrypted solves a different problem:

! What if the database server or highly privileged administrator should not be able to see selected sensitive plaintext values?

How Always Encrypted works

Simplified flow:

EXAMPLE / FLOW

```
Application
    |
    | plaintext:
    | john@example.com
    v
Always Encrypted-enabled client driver
    |
    | encrypts value
    v
Ciphertext
    |
    v
SQL Server
    |
    | stores ciphertext
    v
Database
```

Reading:

EXAMPLE / FLOW

```
SQL Server
    |
    | returns ciphertext
    v
Client driver
    |
    | decrypts using authorized key
    v
Application receives plaintext
```

Always Encrypted keys

Always Encrypted uses two important keys:

EXAMPLE / FLOW

```
Column Master Key
CMK

Column Encryption Key
CEK
```

The CEK encrypts column data.

The CMK protects the CEK.

Conceptually:

EXAMPLE / FLOW

```
Column Master Key
      |
      v
protects
Column Encryption Key
      |
      v
encrypts
Sensitive Column Data
```

Microsoft confirms that column master keys remain under client control in an external trusted key store, while column encryption keys encrypt the protected data.

Deterministic encryption

With deterministic encryption:

EXAMPLE / FLOW

```
same plaintext
+
same key
→ same ciphertext
```

That allows certain equality operations.

For example, searching an encrypted value using equality may be possible through an Always Encrypted-aware driver.

The trade-off is that repeated values produce repeated encrypted representations, leaking some pattern information.

Randomized encryption

Randomized encryption uses randomness so the same plaintext can produce different ciphertext.

Conceptually:

EXAMPLE / FLOW

```
"john@example.com"

Encryption #1
→ A81F...

Encryption #2
→ F913...
```

It offers stronger resistance to pattern analysis but supports fewer traditional query operations.

Microsoft's Always Encrypted tutorial contrasts deterministic equality-search capability with randomized encryption's stronger confidentiality and reduced query functionality.

Secure enclaves

Always Encrypted with secure enclaves extends Always Encrypted by allowing certain protected computations inside a trusted protected memory region.

It can support richer operations such as certain comparisons and pattern matching while maintaining stronger confidentiality boundaries.

When Always Encrypted is useful

Good candidates include extremely sensitive attributes where even database administrators should not routinely see plaintext.

Examples:

EXAMPLE / FLOW

```
National identifiers

Sensitive health data

Bank details

Highly confidential personal data
```

Whether fields such as `PrimaryEmail` need Always Encrypted depends on the real security and compliance requirements.

When not to use Always Encrypted everywhere

Always Encrypted adds complexity:

EXAMPLE / FLOW

```
Key management

Client-driver requirements

Query restrictions

Operational complexity
```

Use it where the threat model justifies that additional protection.

Transparent Data Encryption

Complete definition

Transparent Data Encryption, or TDE, encrypts SQL Server database and transaction-log files at rest so that someone who obtains the physical database files cannot simply attach or restore them and read the data without the required encryption key material.

Microsoft describes TDE as real-time encryption/decryption of database data and log files, protecting data at rest.

Why TDE exists

Imagine somebody steals:

EXAMPLE / FLOW

```
FundraisingDb.mdf

FundraisingDb.ldf

database backup files
```

Without encryption, those files may potentially be restored elsewhere.

TDE protects the files cryptographically.

TDE flow

EXAMPLE / FLOW

```
SQL Server processing
      |
      | plaintext pages in memory
      v
Buffer memory
      |
      | encrypt during disk write
      v
Database files
      |
      v
encrypted at rest
```

When SQL Server reads a page:

EXAMPLE / FLOW

```
encrypted disk page
      |
      v
SQL Server decrypts
      |
      v
usable page in memory
```

Applications do not normally have to change queries.

TDE protects

TDE helps protect:

EXAMPLE / FLOW

```
Database data files

Transaction log files

Backups of the TDE-protected database
```

Microsoft notes that backups of a TDE-enabled database are also encrypted.

TDE does not protect

TDE does **not** prevent an authorized SQL query from reading plaintext.

Example:

SQL

```
SELECT PrimaryEmail
FROM constituent.Constituent;
```

If the user has permission, SQL Server can return the value.

TDE also does not encrypt communication between client and server. Microsoft explicitly distinguishes TDE from communication-channel encryption.

TDE key concept

TDE uses a:

EXAMPLE / FLOW

```
Database Encryption Key
DEK
```

which is protected through the SQL Server encryption hierarchy.

On SQL Server, a certificate in `master` commonly protects the DEK.

Extremely important TDE rule

Back up the certificate and private key protecting the database encryption key.

If you move or restore the encrypted database to another SQL Server without the required certificate/key material, you may be unable to open the database.

Microsoft explicitly warns to back up the certificate and private key immediately after enabling TDE.

Encryption at Rest

Definition

Encryption at rest protects data while it is stored on persistent media.

Examples:

EXAMPLE / FLOW

```
Database files

Transaction logs

Backup files

Storage disks
```

TDE is a major SQL Server at-rest encryption technology.

Think:

EXAMPLE / FLOW

```
Data sitting on disk
→ At rest
```

Encryption in Transit

Definition

Encryption in transit protects data while it is traveling between the application and SQL Server over the network.

For example:

EXAMPLE / FLOW

```
Angular
    |
    v
ASP.NET Core Web API
    |
    | network connection
    v
SQL Server
```

The connection between the Web API and SQL Server may contain:

EXAMPLE / FLOW

```
SQL queries

Passwords/tokens

Constituent information

Gift amounts

Results
```

Network encryption helps stop an attacker on the network from reading that traffic.

TLS

SQL Server uses TLS for encrypted client/server communication.

Conceptually:

EXAMPLE / FLOW

```
ASP.NET Core API
      |
      | encrypted TLS channel
      v
SQL Server
```

Why certificate validation matters

A production configuration should not merely encrypt traffic—it should also validate that the client really connected to the expected SQL Server.

Conceptually:

EXAMPLE / FLOW

```
Encrypt=True
TrustServerCertificate=False
```

with a properly trusted certificate is stronger than bypassing certificate validation.

At Rest vs In Transit

EXAMPLE / FLOW

```
TDE
→ protects stored database files

TLS
→ protects network traffic
```

Neither replaces the other.

Example:

EXAMPLE / FLOW

```
TDE enabled
but TLS disabled
```

means:

EXAMPLE / FLOW

```
Disk protected

Network traffic may still be exposed
```

TDE vs Always Encrypted vs TLS

Technology	Protects
TDE	Database/files/backups at rest
TLS	Network traffic
Always Encrypted	Selected sensitive column values from client through database

Easy memory:

EXAMPLE / FLOW

```
TDE
→ disk

TLS
→ network

Always Encrypted
→ sensitive columns
```

SQL Injection

Complete definition

SQL injection is a security vulnerability that occurs when untrusted input becomes part of executable SQL syntax and changes the meaning of the SQL statement.

The central problem is:

EXAMPLE / FLOW

```
application data
```

gets mixed together with:

EXAMPLE / FLOW

```
SQL code
```

Microsoft describes SQL injection as malicious text being inserted into SQL strings that SQL Server later parses and executes.

Unsafe example

Suppose the application constructs:

C#

```
var sql =
    "SELECT ConstituentId, DisplayName " +
    "FROM constituent.Constituent " +
    "WHERE PrimaryEmail = '" + email + "'";
```

The application is building:

EXAMPLE / FLOW

```
SQL syntax
+
user input
+
SQL syntax
```

That is dangerous.

Normal input

User enters:

EXAMPLE / FLOW

```
john@example.com
```

SQL becomes:

SQL

```
SELECT
    ConstituentId,
    DisplayName
FROM constituent.Constituent
WHERE PrimaryEmail = 'john@example.com';
```

That looks fine.

But the application has no guarantee future input contains only an email address.

Why injection happens

SQL Server receives one SQL string.

It cannot know that certain characters came from:

EXAMPLE / FLOW

```
user input
```

rather than:

EXAMPLE / FLOW

```
developer-written SQL syntax
```

If untrusted input changes the SQL structure, an attacker may:

EXAMPLE / FLOW

```
change WHERE condition

run additional commands

expose data

modify data
```

depending on application permissions.

Parameterized Queries

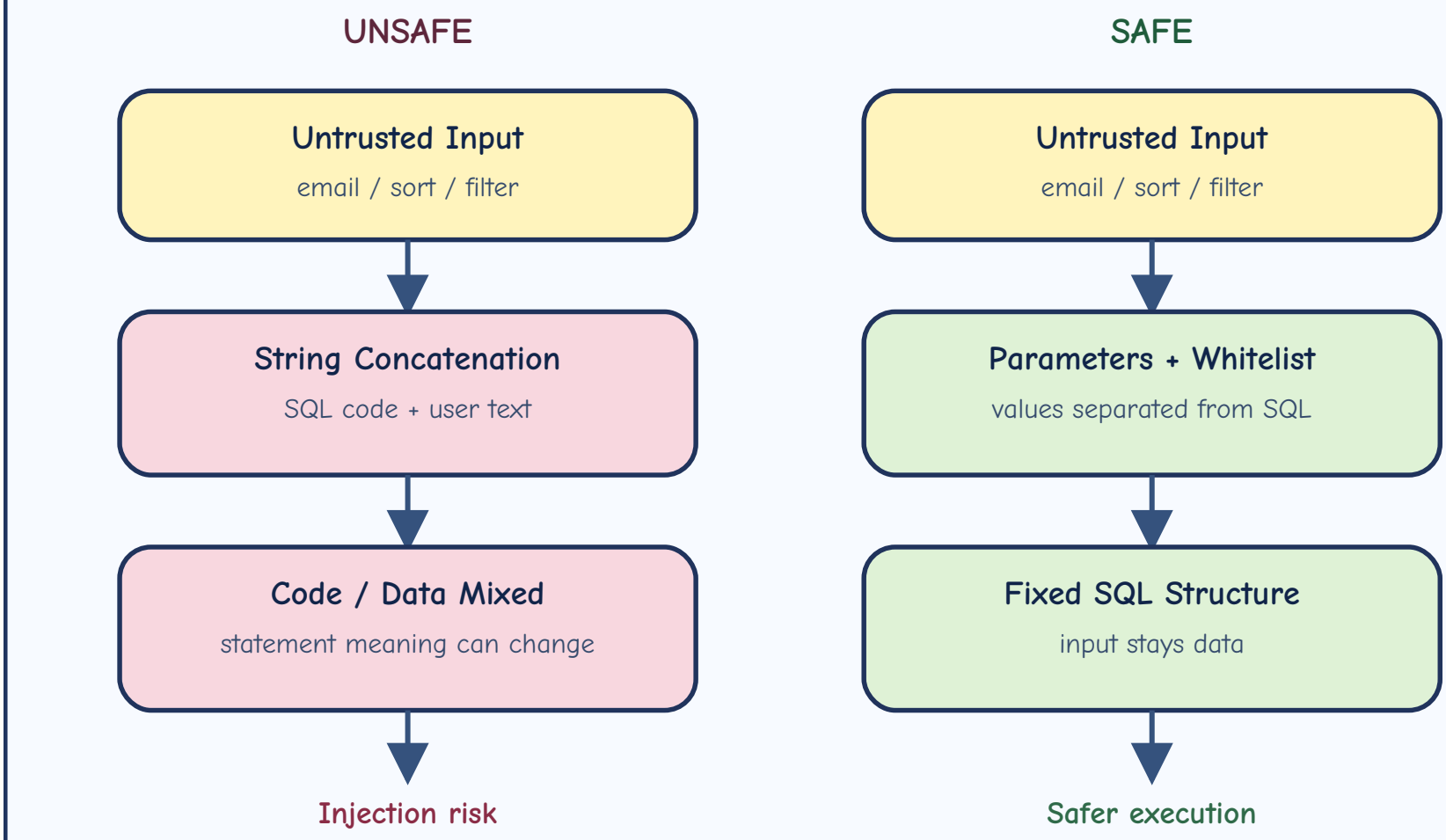
Complete definition

A parameterized query keeps the SQL command structure separate from the input values so that input is treated as data rather than executable SQL syntax.

Unsafe:

DIAGRAM

SQL Injection - Unsafe vs Parameterized Flow



Parameters protect values; identifier choices should come from a trusted whitelist, and the SQL identity should still follow least privilege.

EXAMPLE / FLOW

```
"... WHERE Email = '" + email + "'"
```

Safe concept:

SQL

```
WHERE PrimaryEmail = @Email
```

and send the email value separately.

SQL example

SQL

```
SELECT
    ConstituentId,
    DisplayName,
    PrimaryEmail

FROM constituent.Constituent

WHERE TenantId = @TenantId
  AND PrimaryEmail = @Email;
```

The SQL structure is fixed.

Values are parameters.

C# example

C#

```
using Microsoft.Data.SqlClient;
using System.Data;

using var command = connection.CreateCommand();

command.CommandText = """
    SELECT
        ConstituentId,
        DisplayName,
        PrimaryEmail
    FROM constituent.Constituent
    WHERE TenantId = @TenantId
      AND PrimaryEmail = @Email;
    """;

command.Parameters.Add(
    "@TenantId",
    SqlDbType.BigInt
).Value = tenantId;

command.Parameters.Add(
    "@Email",
    SqlDbType.NVarChar,
    320
).Value = email;
```

Now SQL Server receives:

EXAMPLE / FLOW

```
Command
separately

Parameter values
separately
```

That is the essential SQL injection defense for normal values.

Why parameterization works

The email value is interpreted as:

EXAMPLE / FLOW

```
data
```

rather than:

EXAMPLE / FLOW

```
SQL syntax
```

Even if the value contains quote characters, it remains a value.

Stored procedures and injection

Stored procedures are not automatically safe.

Safe:

```sql
CREATE OR ALTER PROCEDURE constituent.usp_FindByEmail
    @Email NVARCHAR(320)
AS
BEGIN
    SELECT
        ConstituentId,
        DisplayName

    FROM constituent.Constituent

    WHERE PrimaryEmail = @Email;
END;
```

Unsafe:

```sql
DECLARE @Sql NVARCHAR(MAX);

SET @Sql =
    N'SELECT *
      FROM constituent.Constituent
      WHERE PrimaryEmail = '''
    + @Email
    + N'''';

EXEC(@Sql);
```

The unsafe version still concatenates input into SQL.

Dynamic SQL

Dynamic SQL is sometimes necessary.

Use:

```
sys.sp_executesql
```

and parameterize values.

Example:

```sql
DECLARE @Sql NVARCHAR(MAX);

SET @Sql =
N'
SELECT
    ConstituentId,
    DisplayName
FROM constituent.Constituent
WHERE PrimaryEmail = @Email;
';

EXEC sys.sp_executesql
    @Sql,
    N'@Email NVARCHAR(320)',
    @Email = @InputEmail;
```

Microsoft recommends carefully reviewing code that uses **EXEC**, **EXECUTE**, or **`sp_executesql`** and parameterizing dynamically generated SQL.

Dynamic identifiers

Parameters can represent values.

They cannot normally substitute arbitrary SQL identifiers such as:

```
Table name

Column name

ASC

DESC
```

If users can choose a sort column, use a whitelist.

Example:

```
Allowed:
GiftDate
TotalAmount
DisplayName
```

Then map user choice to known identifiers.

Do not accept arbitrary text and append it to SQL.

Least Privilege

Complete definition

Least privilege is the security principle of giving each user, application, service, or administrator only the minimum permissions required to complete its legitimate work, and no additional authority.

This is one of the most important security principles in SQL Server.

Microsoft explicitly recommends minimum required permissions and role separation.

Bad application design

EXAMPLE / FLOW

```
ASP.NET Core API
       |
       v
SQL Login
       |
       v
sysadmin
```

If the application is compromised, the attacker may inherit enough power to:

EXAMPLE / FLOW

```
Drop databases

Create logins

Read unrelated databases

Change server configuration

Disable security
```

The application probably never needed these permissions.

Better design

EXAMPLE / FLOW

```
ASP.NET Core API
       |
       v
FundraisingApiLogin
       |
       v
FundraisingApiUser
       |
       v
FundraisingApiRole
       |
       +-- SELECT Constituent
       +-- INSERT Gift
       +-- INSERT GiftAllocation
       +-- UPDATE Task
       +-- EXECUTE approved procedures
```

No:

EXAMPLE / FLOW

```
sysadmin

db_owner

ALTER DATABASE

CREATE LOGIN
```

unless absolutely required.

Why least privilege reduces risk

Suppose an application has:

EXAMPLE / FLOW

```
INSERT Gift
SELECT Constituent
```

only.

Even if an SQL injection vulnerability appears, the attacker's database capabilities are limited by that application's permissions.

Compare:

EXAMPLE / FLOW

```
SQL Injection
+
sysadmin login
```

with:

EXAMPLE / FLOW

```
SQL Injection
+
narrow custom role
```

Both are bad, but the second has a much smaller possible blast radius.

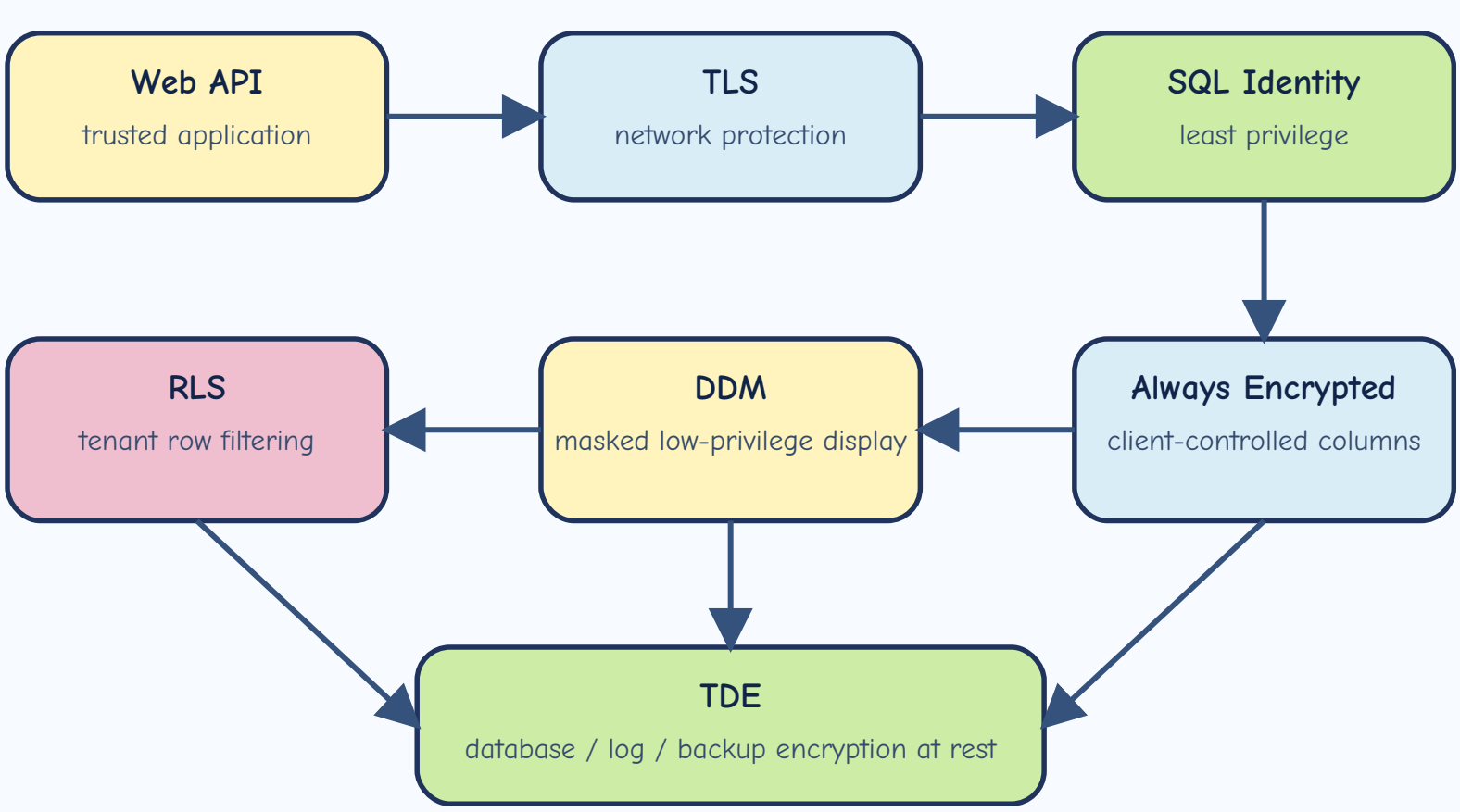
Complete FundraisingDb Security Scenario

Imagine your ASP.NET Core Web API.

A good architecture could look like:

DIAGRAM

Defense in Depth for FundraisingDb



No single control is expected to solve every threat; each layer protects a different boundary.

EXAMPLE / FLOW

```
Authenticated Web API
       |
       v
Dedicated SQL identity
       |
       v
FundraisingApiUser
       |
       v
FundraisingApiRole
       |
       +-- EXECUTE approved procedures
       +-- limited SELECT
       +-- limited INSERT
       +-- limited UPDATE
       |
       v
RLS
Tenant filtering
       |
       v
DDM
Mask low-privilege sensitive output
       |
       v
Always Encrypted
for especially sensitive columns
where required
       |
       v
TDE
protect files at rest
       |
       v
TLS
protect network traffic
```

This is called:

EXAMPLE / FLOW

```
Defense in Depth
```

because the system uses multiple protective layers.

Comparison of the Main Security Features

Feature	Main purpose
Login	Authenticate identity to SQL Server

Feature	Main purpose
Database User	Represent identity inside database
Server Role	Group server-level privileges
Database Role	Group database-level privileges
GRANT	Allow action
DENY	Explicitly block action
REVOKE	Remove permission rule
Schema Permission	Secure group of schema objects
Object Permission	Secure one specific object
RLS	Control which rows are accessible
DDM	Mask how sensitive values appear
Always Encrypted	Encrypt selected sensitive columns from client side
TDE	Encrypt database files at rest
TLS	Encrypt network traffic
Parameterized Query	Keep user input separate from SQL code
Least Privilege	Give only required permissions

Common Security Mistakes

Giving every application db_owner

`db_owner` is extremely broad.

Use custom roles instead.

Giving applications sysadmin

Almost never justified for ordinary business applications.

Granting everything directly to individual users

Use roles.

EXAMPLE / FLOW

```
Users
→ Roles
→ Permissions
```

is easier to maintain and audit.

Using DENY when you only wanted to remove GRANT

Use:

EXAMPLE / FLOW

```
REVOKE
```

to remove the explicit grant.

Granting broad permissions to public

Every login/user belongs to `public`.

Only grant public permissions when you really mean:

EXAMPLE / FLOW

```
everyone.
```

Treating DDM as encryption

DDM masks display values.

It does not cryptographically protect the stored value.

Treating TDE as user access control

TDE protects files.

It does not stop a permitted SELECT.

Treating TLS as disk encryption

TLS protects traffic.

TDE protects stored database files.

Treating Always Encrypted like TDE

They protect different threat models.

Concatenating input into SQL

Use parameters.

Thinking stored procedures automatically eliminate injection

They do not if the procedure concatenates unsafe dynamic SQL.

Trusting client-supplied TenantId for RLS

Tenant context must come from trusted authentication/application logic.

Ignoring key management

Encryption without reliable key backup and key management can become a data-loss problem.

For example, Microsoft explicitly warns that losing the certificate/private key protecting a TDE database can make recovery impossible.

Practical Security Design for FundraisingDb

A simple application role:

```sql
USE FundraisingDb;
GO

CREATE ROLE FundraisingApiRole;
GO
```

Add user:

```sql
ALTER ROLE FundraisingApiRole
ADD MEMBER FundraisingApiUser;
```

Grant narrowly:

```sql
GRANT SELECT
ON OBJECT::constituent.Constituent
TO FundraisingApiRole;

GRANT SELECT, INSERT
ON OBJECT::fundraising.Gift
TO FundraisingApiRole;

GRANT SELECT, INSERT
ON OBJECT::fundraising.GiftAllocation
TO FundraisingApiRole;

GRANT SELECT, UPDATE
ON OBJECT::engagement.Task
TO FundraisingApiRole;
```

The application does not automatically need:

EXAMPLE / FLOW

```
DELETE every table

ALTER TABLE

DROP TABLE

CONTROL DATABASE

sysadmin
```

That is least privilege in practical form.

Interview Questions with Real Answers

What is the difference between authentication and authorization?

Authentication verifies the identity of the user, application, or service connecting to SQL Server, while authorization determines what that authenticated identity is allowed to do.

For example, `FundraisingApiLogin` may successfully authenticate to SQL Server. After authentication, SQL Server checks whether the corresponding database user or role has permission to `SELECT constituent.Constituent` or `INSERT` into `fundraising.Gift`.

The easy memory rule is:

EXAMPLE / FLOW

```
Authentication
→ Who are you?

Authorization
→ What can you do?
```

What is a SQL Server login?

A login is a **server-level principal used to authenticate to a SQL Server instance**.

For example:

EXAMPLE / FLOW

```
FundraisingApiLogin
```

allows your ASP.NET Core application to establish an identity at the SQL Server instance level.

The login alone does not automatically give access to every table. It is normally mapped to a database user that receives database permissions.

What is a database user?

A database user is an **identity inside one specific database**.

For example:

EXAMPLE / FLOW

```
FundraisingApiLogin
```

may map to:

EXAMPLE / FLOW

```
FundraisingApiUser
```

inside `FundraisingDb`.

The easiest distinction is:

EXAMPLE / FLOW

```
Login
→ server identity

User
→ database identity
```

What is a server role?

A server role is a group of server-level permissions.

Its members are generally logins.

For example, `sysadmin` gives unrestricted SQL Server administration capability, while newer SQL Server roles such as `##MS_ServerStateReader##` can provide narrower administrative capabilities.

For normal application accounts, avoid powerful server roles unless they are truly required.

What is a database role?

A database role groups database users and gives them common database-level permissions.

For example, rather than granting Gift permissions to five application users individually, you can create:

EXAMPLE / FLOW

```
GiftProcessingRole
```

grant permissions once, and make those users members of the role.

This makes administration easier and supports least privilege.

What is the difference between GRANT, DENY, and REVOKE?

`GRANT` explicitly allows an action.

```sql
GRANT SELECT
ON fundraising.Gift
TO ReportingRole;
```

DENY explicitly prevents the action.

```sql
DENY DELETE
ON fundraising.Gift
TO ReportingRole;
```

REVOKE removes the explicit permission decision.

```sql
REVOKE SELECT
ON fundraising.Gift
FROM ReportingRole;
```

A revoked user might still obtain the permission from another role, while an applicable DENY normally overrides ordinary grants.

What is the difference between schema and object permissions?

A schema permission applies more broadly to relevant objects inside a schema.

Example:

```sql
GRANT SELECT
ON SCHEMA::fundraising
TO ReportingRole;
```

can cover multiple fundraising objects.

An object permission applies only to one object.

```sql
GRANT SELECT
ON OBJECT::fundraising.Gift
TO GiftReportingRole;
```

Use object permission when access should be narrow. Use schema permission when the schema itself is an intentional security boundary.

What is Row-Level Security?

Row-Level Security automatically filters or blocks rows based on a security predicate.

For a multi-tenant database, RLS can enforce:

EXAMPLE / FLOW

```
Tenant 1
→ Tenant 1 Gifts only

Tenant 2
→ Tenant 2 Gifts only
```

even if both applications query the same `fundraising.Gift` table.

RLS commonly uses an inline table-valued predicate function attached through a security policy.

What is the difference between an RLS filter predicate and block predicate?

A filter predicate controls which rows a user can see or act on.

A block predicate prevents writes that violate the security rule.

For example:

EXAMPLE / FLOW

```
Tenant 1 reads Tenant 2 row
→ filter hides it

Tenant 1 tries to insert TenantId 2
→ block predicate can reject it
```

What is Dynamic Data Masking?

Dynamic Data Masking changes how sensitive values appear in query results for users who do not have sufficient permission.

For example:

EXAMPLE / FLOW

```
Stored:
john.smith@example.com

Displayed:
jXXX@XXXX.com
```

The stored value remains unchanged.

Therefore DDM is a presentation-level protection, not encryption.

What is TDE?

Transparent Data Encryption encrypts SQL Server database and log files at rest.

Its purpose is to protect data when physical files, disks, or backups are stolen or accessed outside SQL Server.

TDE is transparent to normal applications because SQL Server decrypts pages when they are loaded into memory.

Does TDE stop an authorized user from SELECTing sensitive data?

No.

If the user has permission to query the column:

```sql
SELECT PrimaryEmail
FROM constituent.Constituent;
```

SQL Server can return the plaintext.

TDE protects storage, not database authorization.

What is Always Encrypted?

Always Encrypted encrypts selected sensitive columns using client-controlled keys.

The SQL Server Database Engine normally stores ciphertext while the application driver handles encryption/decryption.

It is particularly useful when highly privileged database administrators should not automatically have access to selected plaintext values.

What is the difference between TDE and Always Encrypted?

TDE protects the entire database storage layer.

Always Encrypted protects selected sensitive columns from the client side.

Think:

```
TDE
→ protect files

Always Encrypted
→ protect selected values
```

TDE is transparent to SQL Server queries. Always Encrypted requires compatible client drivers and has query limitations.

What is encryption at rest?

Encryption at rest protects data stored on persistent storage.

Examples:

```
Database files

Log files

Backup files
```

TDE is a major SQL Server at-rest encryption technology.

What is encryption in transit?

Encryption in transit protects data while it travels over the network between the application and SQL Server.

SQL Server uses TLS for this purpose.

The main idea is:

EXAMPLE / FLOW

```
Application
→ encrypted network connection
→ SQL Server
```

What is SQL injection?

SQL injection occurs when untrusted input becomes part of executable SQL syntax.

It commonly happens when applications construct SQL strings using concatenation.

Example of a dangerous pattern:

EXAMPLE / FLOW

```
SQL code
+
user input
+
SQL code
```

The primary defense for values is parameterized SQL.

Why are parameterized queries safer?

Parameterized queries send:

EXAMPLE / FLOW

```
SQL command
```

and:

EXAMPLE / FLOW

```
input values
```

separately.

SQL Server treats parameter values as data instead of interpreting them as additional SQL syntax.

Example:

```
SQL
WHERE PrimaryEmail = @Email
```

is much safer than concatenating the email directly into the SQL string.

Can stored procedures still have SQL injection?

Yes.

Stored procedures are safe only if they handle input safely.

A procedure that builds dynamic SQL by concatenating:

```
EXAMPLE / FLOW
@Email
```

into the SQL text can still be vulnerable.

When dynamic SQL is required, use parameterized `sys.sp_executesql` for values.

What is least privilege?

Least privilege means giving a user or application only the minimum permissions necessary to perform its responsibilities.

For example, your API might need:

```
EXAMPLE / FLOW
SELECT Constituent
INSERT Gift
INSERT GiftAllocation
UPDATE Task
```

It normally does not need:

```
EXAMPLE / FLOW
sysadmin
db_owner
DROP DATABASE
CREATE LOGIN
```

Least privilege reduces the damage that can occur if an account, application, or query is compromised.

Final Security Mental Model

EXAMPLE / FLOW

```
USER / APPLICATION
        |
        v
AUTHENTICATION
Who are you?
        |
        v
LOGIN
Server-level identity
        |
        v
DATABASE USER
Database-level identity
        |
        v
ROLE
Group permissions
        |
        v
GRANT / DENY / REVOKE
        |
        v
SCHEMA / OBJECT PERMISSIONS
        |
        v
DATA ACCESS
        |
        +-----------------------------+
        |                             |
        v                             v
ROW / DISPLAY SECURITY          ENCRYPTION
        |                             |
      RLS                           Always Encrypted
      DDM                           TDE
                                    TLS
```

And application security adds:

EXAMPLE / FLOW

```
Untrusted Input
      |
      v
Parameterized Query
      |
      v
Least-Privilege SQL Identity
      |
      v
FundraisingDb
```

Final Memory Rules

EXAMPLE / FLOW

Authentication
→ Verify who is connecting

EXAMPLE / FLOW

Authorization
→ Decide what that identity may do

EXAMPLE / FLOW

Login
→ SQL Server instance identity

EXAMPLE / FLOW

User
→ Database identity

EXAMPLE / FLOW

Server Role
→ Server-level permission group

EXAMPLE / FLOW

Database Role
→ Database-level permission group

EXAMPLE / FLOW

GRANT
→ Allow

EXAMPLE / FLOW

DENY
→ Explicitly block

EXAMPLE / FLOW

REVOKE
→ Remove the explicit rule

EXAMPLE / FLOW

Schema Permission
→ Access across a schema

EXAMPLE / FLOW

Object Permission
→ Access one table/view/procedure/etc.

EXAMPLE / FLOW

RLS
→ Which rows can I access?

EXAMPLE / FLOW

DDM
→ How should a sensitive value appear?

EXAMPLE / FLOW

Always Encrypted
→ Protect selected sensitive column values
using client-controlled keys

EXAMPLE / FLOW

TDE
→ Encrypt SQL Server files at rest

EXAMPLE / FLOW

TLS
→ Encrypt SQL Server network traffic

EXAMPLE / FLOW

Parameterized Query
→ Separate SQL code from input data

EXAMPLE / FLOW

Least Privilege
→ Give only the permissions genuinely required

The most important overall rule is:

! SQL Server security should be layered. Authenticate identities correctly, map them to database users, assign permissions through narrowly scoped roles, use GRANT/DENY/REVOKE carefully, control row visibility with RLS where appropriate, mask sensitive display values with DDM, use Always Encrypted when highly sensitive columns need client-controlled protection, use TDE for database files, use TLS for network traffic, parameterize every untrusted value, and always follow least privilege.